

Motýlí klasifikátor přes machine learning

Autor: Martin Morávek

Kontaktní údaje: martin.moravek@spsejecna.cz

Datum vypracování: 18.4. 2026

Název školy: [Střední průmyslová škola elektrotechnická, Praha 2, Ječná 30](#)

Jedná se o školní projekt

Specifikace požadavků:

Projekt implementuje klasifikátor fotografií motýlů využívající hluboké učení (deep learning). Program dokáže na základě fotografie automaticky určit druh motýla ze seznamu naučených druhů.

Architektura aplikace:

Aplikace se skládá z 3 hlavních skriptů, které mají jasně oddělené funkce. Skript `Analyser.py` slouží k načtení dat, předzpracování, sestavení modelu, trénink, vyhodnocení a uložení modelu. `Main.py` dkouží k načtení uloženého modelu a klasifikace nových fotografií. `Main.py` nic netrénuje — pouze používá hotový model. `AppUI.py` slouží k zobrazení jednoduché python aplikace. Jádrem modelu je konvoluční neuronová síť `EfficientNetV2S` předtrénovaná na datasetu ImageNet (14 milionů obrázků, 1000 kategorií). Tato síť slouží jako extraktor příznaků. V první fázi tréninku jsou její váhy zmrazeny, ve druhé fázi jsou rozmrazeny vrchní vrstvy a doladěny na motýlí data. Výpočty jsou prováděny v datovém typu `float16` pro přibližně dvojnásobné zrychlení na moderních GPU (Turing, Ampere, Ada). Váhy modelu jsou uchovávány v `float32` pro zachování přesnosti. `LossScaleOptimizer` zabraňuje podtečení gradientů.

Běh aplikace:

Tréninková část (butterfly_classifier_v2.py)

1. Spuštění skriptu
2. Načtení fotografií ze složky DATA_DIR, extrakce názvů druhů z názvů souborů
3. Rozdělení dat na tréninková (75 %), validační (15 %) a testovací (10 %)
4. Výpočet vah tříd pro vyrovnaní nevyváženého datasetu
5. Sestavení tf.data pipeline — streamování obrázků z disku po dávkách
6. Definice plánu učebního kroku (WarmupCosineDecay)
7. Sestavení modelu (augmentace + EfficientNetV2S + klasifikační hlava)
8. Fáze 1: trénink pouze klasifikační hlavy (páteřní síť zmrazena), max. 80 epoch
9. Fáze 2: doladění vrchních 40 vrstev páteřní sítě, max. 50 epoch
10. Vyhodnocení na testovacích datech — přesnost, ztrátová funkce, klasifikační zpráva
11. Uložení modelu do souboru .keras

Inferenční část (butterfly_inference.py)

1. Spuštění skriptu
2. Sestavení architektury modelu z kódu
3. Načtení uložených vah ze souboru .keras
4. Načtení fotografie zadané uživatelem
5. Provedení 8 průchodů s Test-Time Augmentation (TTA)
6. Průměrování výstupních pravděpodobností
7. Výpis top 3 nejpravděpodobnějších druhů s procentuální jistotou
8. Ukončení

appUI.py

1. Spuštění aplikace (app.py)
2. zobrazení okna aplikace
3. Výběr libovolné fotografie v zařízení
4. Vyhodnocení výsledků
5. Ukončení aplikace

Import/export

Export

Po dokončení tréninku je model automaticky exportován do model.keras.

Import

Aplikace nepodporuje import dat v jiném formátu než JPEG a PNG. Fotografie musí být umístěny kdekoli v zařízení, na kterém testování probíhá.

Konfigurace aplikace

Konfigurace aplikace je uložena přímo v hlavičce tréninkového skriptu `analyser.py` v sekci označené jako `CONFIGURATION`. Není použit žádný externí konfigurační soubor.

Použitá rozhraní, knihovny a závislosti

Programovací jazyk:

- Python 3.11 nebo novější — Keras 3.x nepodporuje Python 3.10 a starší

Knihovny:

- TensorFlow – Framework pro strojové učení, tf.data pipeline (<https://www.tensorflow.org>)
- Keras – API pro stavbu a trénink neuronových sítí (<https://keras.io>)
- scikit-learn – Rozdělení dat, kódování štítků, váhy tříd, klasifikační zpráva (<https://scikit-learn.org>)
- NumPy – Pro operace s poli (<https://numpy.org>)
- h5py – keras používá pro uložení modelu (<https://www.h5py.org>)

Právní a licenční aspekty

- Projekt je studentský
- Neobsahuje licencovaný komerční software
- Předtrénované váhy EfficientNetV2S pochází z veřejně dostupného datasetu ImageNet a jsou poskytnuty pod licencí Apache 2.0 prostřednictvím knihovny Keras
- Všechny použité knihovny (TensorFlow, Keras, scikit-learn, NumPy, Matplotlib) jsou open-source a dostupné zdarma
- Fotografie použité pro trénování byly všechny vlastnoručně pořízené
- Projekt je chráněn pod licencí MIT

Instalace a spuštění

Požadavky:

- Python 3.11 nebo novější
- Operační systém Windows / Linux
- Doporučeno: NVIDIA GPU s compute capability 7.0+ (Volta / Turing / Ampere / Ada) pro smíšenou přesnost

Postup:

1. Nainstalovat Python (pokud není nainstalovaný)
2. Rozbalit projekt
3. Stáhnout knihovny tensorflow, keras, scikit-learn, NumPy, Matplotlib
4. Spustit v konzoli v složce, kde je stažený projekt příkaz `python main.py` nebo spustit `main.py` v Python IDE jako je Pycharm

Chybové stavy a jejich řešení

Chybový stav	Příčina	Řešení
Nulová přesnost (~1 %) po celou dobu tréninku	a) Dvojitá normalizace obrázků (dělení 255 dvakrát) b) Chybějící LossScaleOptimizer při smíšené přesnosti	a) Odstranit /255 z funkce preprocess() — páteří síť normalizaci provádí interně b) Obalit Adam optimizer třídou LossScaleOptimizer
Divoké výkyvy přesnosti	Příliš malý BATCH_SIZE vůči počtu druhů — každá dávka pokrývá pouze malou část druhů	Zvýšit BATCH_SIZE na 64 nebo 128
Ztráta NaN po několika epochách	Příliš vysoký učební krok v kombinaci s float16	Snížit P1_LR na 5e-4; ověřit přítomnost LossScaleOptimizer
FileNotFoundError při spuštění	Proměnná DATA_DIR neukazuje na existující složku	Zkontrolovat, zda složka butterflies/ existuje ve stejném adresáři jako skript
Žádné druhy nebyly detekovány	Názvy souborů neodpovídají konvenci NázevDruhu_cokoliv.jpg	Přejmenovat soubory tak, aby název druhu předcházel prvnímu podtržítku
Chyba nedostatku paměti GPU (OOM)	BATCH_SIZE nebo IMG_SIZE příliš velké pro dostupnou VRAM	Snížit BATCH_SIZE na 32; nebo snížit IMG_SIZE na (300, 300)
Chyba serializace (EagerTensor) při ukládání modelu	Augmentační vrstvy jsou mimo graf modelu	Augmentace musí být součástí grafu modelu — nepřesouvat ji do tf.data pipeline
TypeError při load_model() — version mismatch	Model byl uložen jinou verzí Keras, než je nainstalována lokálně	Zajistit shodnou verzi Keras v obou prostředích (Colab i lokálně). Keras 3.x vyžaduje Python 3.11+
Špatné predikce inference (jiný druh než skutečný)	Seznam CLASS_NAMES v butterfly_inference.py není ve správném abecedním pořadí	Zkopírovat seznam přesně z výstupu tréninku — LabelEncoder řadí druhy abecedně

Diagram

Nachází se v `AnalyserDiagram.mermaid` a `TestDiagram.mermaid`

Sít a externí služby

- Aplikace nevyžaduje síťové připojení při tréninku ani při inferenci
- Předtrénované váhy EfficientNetV2S jsou při prvním spuštění automaticky staženy z internetu prostřednictvím knihovny Keras a uloženy do lokální cache (`~/.keras/`). Při dalších spuštěních se používají lokálně uložené váhy
- Nevyžaduje webový server ani databázový server
- Nevyžaduje žádné API klíče ani externí účty

Hardware a další služby

- Běžný PC nebo laptop s operačním systémem Windows nebo Linux
- Python 3.11+
- Minimálně 8 GB RAM pro dataset 10 000 obrázků
- Minimálně 10 GB volného místa na disku (fotografie + model + předtrénované váhy)
- NVIDIA GPU s compute capability 7.0 nebo vyšší a minimálně 6 GB VRAM pro BATCH_SIZE 64 při rozlišení 384×384
- 16 GB RAM nebo více
- SSD disk pro rychlejší načítání fotografií z disku během tréninku

Alternativně lze použít Google Colab (Keras 3.13 je v Colab dostupný v aktuální verzi). Na CPU je trénink funkční, ale mnohem pomalejší — pro dataset 10 000 obrázků může být doba tréninku v řádu desítek hodin namísto jednotek hodin na GPU.